Final review: Q&A defence sheet

Context-Aware Personal AI Assistant · BCSE497J Project-I · Nischay N, Tejas K M, Tenidhar Reddy Chintham · Guide: Prof. Mary Mekala A. Every number below comes from results/final in the repository.

1. The 30-second pitch and the numbers to know

Pitch. "Our assistant lets anyone plug in any model, local or cloud. Each model is scored on a public benchmark, per type of task, with a named score: the Routing Utility Score. The router sends each question to the model with the best score for that task type. Memory, personal files and the scorecards never leave the user's computer, and any question that carries personal context is kept on local models. On 200 unseen public test tasks, routing matched the most expensive model's accuracy at 95% lower cost."

Claim	Number
Routed (balanced) accuracy	89.5% (179/200), 95% CI 84.5%–93%
Always Claude Sonnet 5.5	89% (178/200) at $2.506 per 1,000 questions
Routed cost	$0.127 per 1,000 (₹11.1), 95% cheaper than Sonnet
Routed vs Sonnet	McNemar exact p = 1.0: no measurable difference
Routed vs Gemini 3.1 Flash-Lite	89.5% vs 86%, p = 0.02
Routed vs best local model	89.5% vs 48% (llama3.2-3b), p < 0.001
Local-first cascade	88%, 28.5% of questions answered on the laptop, $0.105 per 1,000, p = 0.79 vs Sonnet
Oracle (perfect hindsight)	92.5%: the ceiling any router could reach on these models
Intent classifier, unseen public prompts	97% (194/200), 95% CI 93.6%–98.6%
Memory and documents (build D)	recall 28/30; document answers 32/36; baseline 0 on both
Privacy check, every cloud model available	0 of 54 requests with personal context left the laptop
Total paid API spend	about $1.87 (hard $5 cap in code; Gemini on its free tier)

One sentence to remember: same accuracy as the best cloud model, a fraction of the cost, and private data never leaves the laptop.

2. The three Review-2 criticisms and what changed

Q: "The graphs and diagrams were AI-generated and inaccurate (the actor graph)."

- Every diagram is produced by a script in the repository (scripts/make*diagrams.py*, *scripts/diagrams*uml.py) and every chart by scripts/make_figures.py from the saved results. Anyone can regenerate them.
- Each uses a standard notation: UML 2.5 use case, class, sequence and activity diagrams (PlantUML); data flow diagrams in Yourdon–DeMarco notation (circles, open-ended stores, named flows).
- The class diagrams are parsed from the source code with Python's ast module, with attribute types and operation signatures. If a class changes and the diagram's type table is not updated, the build fails, so the diagram cannot drift from the code.
- Use cases: two actors that exist in the system (User; Contributor, a specialised User who registers, calibrates and evaluates models) and two secondary actors (local runtime, cloud API). Every use case is a control in the app or a command. «extend» arrows carry the condition that enables them (for example [build B or D] for memory).

- If asked “did you use AI for these?”: “AI tools helped write the drawing scripts, but every diagram is computed from our code and every chart from our measured results, and the scripts are in the repository.”

Q: “What is the score name? How are the models graded?”

- The **Routing Utility Score (RUS)**, defined in docs/SCORING.md and report section 4.3.
- Each model has a **scorecard per task type**: tasks tried, tasks correct, accuracy, its **95% Wilson interval**, median latency and measured cost per question (real token counts × list price).
- RUS = 0.6·A + 0.2·(1 − C/$0.002) + 0.2·(1 − L/30 s), with A the **lower** bound of the Wilson interval. Weights are adjustable; presets balanced, quality-first, cost-saver.
- Correctness is decided by code, never by an LLM: code runs against the benchmark's unit tests, maths answers are compared as numbers, factual answers against the accepted answers, instructions with the IFEval checks.

Q: “Why does the app show 100% on direct questions?”

- That 100% came from a random split of our own templated prompts: near-copies sat in both training and test. Grouping by template gave 90%.
- Now the classifier is tested on **200 public benchmark prompts it never saw**. It is trained on our prompts plus the public **calibration** prompts (a test checks the test prompts are never used). Result: **97%** (95% CI 93.6%–98.6%). Before that change it scored 83% and recognised only 18/40 writing prompts.
- The phrasing study still shows 50/50 on direct phrasings, but it is reported as an interval, 92.9%–100%: with 50 prompts we can only say “at least about 93%”. Casual 47/50, verbose 49/50.

3. Scoring questions

Q: What exactly is the Routing Utility Score?

RUS = wA·A + wC·(1 − min(C ÷ Cmax, 1)) + wL·(1 − min(L ÷ Lmax, 1)). A is the lower end of the model's 95% Wilson interval for that task type, C its mean measured cost per question, L its median latency; wA, wC, wL are the user's weights. Cmax = $0.002 and Lmax = 30 s are budgets. Highest RUS wins; ties go to the cheaper, then faster model.

Q: Why the Wilson lower bound and not plain accuracy?

Because 40/40 and 4/4 are both “100%” but mean very different things. The Wilson lower bound for 40/40 is 91.2%; for 4/4 it is 51%. Wilson is used because it stays valid near 0% and 100%, where the textbook normal interval breaks.

Q: Walk me through one routing decision (coding, balanced weights).

Model	Correct / 40	A	Cost / question	Latency	RUS
Claude Haiku 5.5	39	0.871	$0.00013	2.2 s	0.895
Gemini 3.1 Flash-Lite	35	0.739	$0.00012	1.3 s	0.822
Claude Sonnet 5.5	40	0.912	$0.00162	2.1 s	0.772
qwen2.5-coder-3b	22	0.398	$0	5.0 s	0.606

Claude Haiku 5.5 wins: Sonnet is slightly more accurate but costs 81% of the budget, so its cost term almost vanishes. With quality-first weights Sonnet wins instead.

Q: Where do cost and latency come from?

Measured. Every call records input and output tokens from the provider's response; cost = tokens × list price from models.yaml. Latency is wall-clock time per answer on our laptop. Local models cost $0.

Q: How does someone add a new model?

Add an entry to models.yaml (provider, model id, price, local or not, vision or not) or run python -m assistant add-model, then python -m assistant calibrate --model <name>. Calibration runs the 200 calibration tasks and writes the scorecard. Providers: Ollama, any OpenAI-compatible API (OpenAI, Groq, Together, LM Studio, vLLM), Anthropic, Gemini.

Q: Why 0.6 / 0.2 / 0.2?

A default, not a claim. The weight sweep shows the trade-off: accuracy stays at 89.5% for cost weights up to 0.6; at 0.7 it is 85.5% with 20% local; at 0.8, 63% of questions go local and accuracy falls to 65.5%. The user picks the point; the app has a slider and three presets.

4. Results questions

Q: Routing beats Sonnet by only 0.5 points. Is that real?

No, and we do not claim it is. McNemar's exact test gives p = 1.0: they disagree on only 11 tasks. The claim is **equal accuracy at 95% lower cost**.

Q: Then why not always use Claude Haiku?

Fair point, and the report says so: Haiku alone scores 88.5% (p = 0.73 vs routing). Routing found that from the scorecards, sends Gemini the questions where Gemini is better and cheaper (74 of 200), and changes automatically when a new model is registered. Routing is cheaper than Haiku alone ($0.127 vs $0.169 per 1,000).

Q: Per task type? (correct out of 40)

Task type	Routed	Sonnet	Best local (llama3.2-3b)
Coding (MBPP)	39	40	20
Maths (GSM8K)	39	39	33
Factual (NQ)	24	25	14
Writing (IFEval)	37	34	29
Images	40	40	0

Q: Why is factual search only about 60%?

Natural Questions is closed-book and its accepted answers are short Wikipedia spans from 2018, so a correct but differently worded or more recent answer can fail. Every model scores low here (Sonnet 62.5%). The assistant's RAG handles the user's own documents; the benchmark deliberately tests without retrieval.

Q: What does the local-first cascade add?

It keeps coding and maths on the laptop when the answer can be checked without knowing the right answer: generated code must pass the assert written in the user's own prompt; two different local models must agree on the number. Otherwise it escalates to the RUS choice. Result: 28.5% of all questions answered on the laptop, 88% accuracy, 96% cheaper than Sonnet. Kept answers were right 54 times in 57. The price is latency, because the local attempt runs first.

Q: What is the oracle?

For each test task, the cheapest model that answered it correctly, with hindsight: 92.5% at $0.111 per 1,000 with 69% local. No real router can know this in advance; the gap to 89.5% is the room left for a better router.

5. Method and honesty questions

Q: How do you know you did not tune on the test?

The 400 tasks are split before anything runs: 40 calibration and 40 test per type, seed 2026. Scorecards and the classifier's public prompts come only from the calibration split. Every accuracy reported is on the 200 test tasks.

Q: Did you change the grading after seeing results?

- **Code:** usage examples that crash (an input() call, a wrong example) failed correct functions. The grader now keeps the functions defined before the crash and runs the benchmark's tests on them. It also fixes the hash seed, because code such as max(set(s), key=s.count) breaks ties by hash order and would otherwise pass on one run and fail on the next.
- **Factual answers:** a manual audit found correct answers marked wrong ("Rachel Bilson" when the accepted answer is "Rachel Sarah Bilson"; "May 10, 1940" vs "10 May 1940"). A word-overlap rule was added, with guards (numbers must match; the question's own words do not count).
- **Sensitivity:** with the original strict grader routing scores 87.5% and Sonnet 87.5%. Same conclusion.

Q: Some Claude answers were empty. What happened?

Claude 5.5 models think before answering and the thinking used up the output budget. 88 such answers were re-run with a 4,096-token floor (39 then passed). Every truncated answer was retried, pass or fail.

Q: Would you get the same numbers again?

Model	Run 1	Run 2	Same pass/fail	McNemar p
Claude Haiku 5.5	177/200	173/200	96%	0.29
Claude Sonnet 5.5	178/200	178/200	99%	1.00
Gemini 3.1 Flash-Lite	170/198	170/198	100%	1.00

Wording changes between runs (the Claude 5.5 models do not accept a temperature setting), but the pass/fail mark is stable and no change is significant. Local models run at temperature 0 with a fixed seed.

Q: Can someone reproduce it?

Yes: run_final.command reruns every step. Raw answers are saved in results/final/raw.jsonl, so the analysis and every chart can be regenerated without calling any model.

6. Privacy, open source and architecture

Q: What exactly stays on the computer?

One local folder (ASSISTANT_HOME, default ~/.context-assistant): the memory database (SQLite: turns, facts, summaries), the document index, the scorecards, the cost ledger and the egress log.

Q: How do you guarantee Claude never sees my memory or files?

By rule, in the router, before any model is chosen: if a question carries private context (a recalled memory, a retrieved passage or earlier turns), only models marked local are eligible. Cloud models are not scored for that question. The app marks every such answer "never left this computer".

Q: Can you prove it?

We measured it. The full build ran the memory benchmark with every model available, including Gemini, Claude Haiku and Claude Sonnet: 0 of 54 requests went to a cloud model, recall stayed at 28/30, and the egress log for that run is empty.

Q: Do memory and documents actually help? (the ablation)

Build	Memory recall	Document Q&A	Made-up answers	Points to a source
Baseline	0/30	0/36	0/36	0/36
Model A	0/30	0/36	5/36	0/36
Model B	30/30	0/36	0/36	0/36
Model C	0/30	31/36	1/36	34/36
Model D	28/30	32/36	1/36	35/36

Each switch does its job and D keeps both. Model B's 30/30 is reported with its interval (88.6%–100%). Answers built from documents now list the documents they used even when the model forgets the [n] markers (build D: 35/36 point to a source, 31 with the model's own markers).

Q: Why TF-IDF + SVM for the intent classifier and not an LLM?

It runs on every question, instantly, on the laptop, for free, and without sending the question anywhere before routing. It still misroutes 6 of 200 public test prompts, stated as a limitation.

Q: What is open source about it?

The whole repository is MIT-licensed on GitHub: code, scorecards (plain JSON), benchmark builder, graders, analysis and diagram scripts. Anyone can score their own models on the same tasks.

7. Hard questions and honest limitations

- **"200 test tasks is small."** Agreed: every number carries a 95% interval (about ±5 points) and every comparison a McNemar test. The scripts take --per-intent and resume, so the benchmark scales with more API budget.
- **"Always using Haiku would do as well."** True on this benchmark (88.5% vs 89.5%). Routing found it without anyone knowing in advance, is cheaper, and adapts when models change.
- **"The classifier makes mistakes."** 6 of 200 public prompts are misrouted, mostly factual questions phrased like arithmetic. Misrouting barely costs accuracy because the strong models handle every type.
- **"Your local models are weak."** They are 1B–3B models on an 8 GB laptop; alone the best reaches 48%. That is why the local-first step keeps only answers it can check (54/57 right). 7B–8B models would raise the on-device share with no code change.
- **"The image tasks are too easy."** They are generated with known answers to test vision routing, not frontier vision; cloud models score 100%, moondream 17.5%. Stated as a limitation.
- **"Your factual grader is lenient."** Both graders are reported; the conclusion holds under the strict one.
- **"Prices and models change."** Prices are read from models.yaml; rerunning calibrate refreshes a scorecard.
- **"OpenAI is missing."** The OpenAI-compatible backend is implemented and GPT-5 mini is registered, but it was not scored because the account had no API credit.
- **"Normal questions still go to the cloud."** Yes, by design; what never leaves is memory, documents, history and scorecards. Every cloud request is in the egress log.
- **"Did you use AI to build this?"** AI coding tools were used as assistants; the design, experiments and every number are ours and reproducible from the repository.